ENGINEERING MATURITY

# HORIZON GRID

Engineering History — Development Journey and Bug History

**Version:** 0.2.4
**Documentation prepared:** 2026-08-21

PREPARED FOR EVALUATION

# Table of Contents

# The Development Journey

## Before Version Control

The product already existed as a working prototype before formal version control began — the earliest QA documents in this repository ( `BACKEND_TEST_ROOT_CAUSE_REPORT.md` , `API_CONFIGURATION_FIX_REPORT.md` , `ADMIN_RBAC_QA_REPORT.md` , `RUNTIME_PROVIDER_IMPLEMENTATION_REPORT.md` ) describe a system with a working Setup Wizard, a database-backed RBAC console, and a runtime-mutable AI/provider configuration layer, none of which are attributed to any commit in this repository's history. That earlier phase is not independently documented here, and this narrative does not attempt to reconstruct it — verifiable history begins at git's own root commit, `9bee96c` , "Initial commit: HORIZON GRID threat intelligence platform," recorded 2026-08-18 21:37.

## The Initial Commit

By the time `9bee96c` was recorded, the platform was already substantial, not a skeleton. Reading the committed tree directly: `README.md` already opens with the name "HORIZON GRID" and the tagline "Every Signal. One Operational Picture."; the deterministic scoring engine ( `backend/app/scoring/engine.py` ) and the dashboard aggregation module ( `backend/app/core/dashboard.py` ) both trace back to this exact commit with `git log --follow` — they were not added later. The frontend's own `package.json` , however, still carried the internal name `"ioc-intel-platform-frontend"` at this point, a naming inconsistency that outlived several later releases. This single commit is the entire starting baseline for every subsequent phase described below; there is no earlier commit to compare it against.

## The Adversarial Audit

The next five commits, all recorded within about ninety minutes of each other late on 2026-08-18, form a self-escalating hardening cycle rather than a single fix.

`c222d2f` ("Add GitHub repo scaffolding: README, SECURITY.md, CI, templates; fix real issues found along the way") added the GitHub-facing scaffolding — issue templates, a PR template, CODEOWNERS, and three CI workflows — grounded in a fresh source-code audit that counted 17 IOC providers plus one OSINT crawler and 6 AI backends at that point. The same commit closed real dependency advisories ( `python-jose` 3.3.0→3.4.0, `cryptography` 43.0.1→49.0.0, `python-multipart` 0.0.9→0.0.31) and fixed a genuinely broken `frontend/.eslintrc.json` — Next.js's linter had no config at all and was silently unusable, and adding one surfaced two real `react/no-unescaped-entities` errors in `NetworkAccessPanel.tsx` .

`54362b2` is a CI-only bugfix: a new Docker-based integration test job ran the test suite against a genuinely empty database for the first time (rather than the developer's own persistent dev database, which already had an admin account), and `test_role_downgrade_immediately_revokes_the_old_roles_access` failed because promoting the test's lone user to admin made them the sole active admin — demoting them then tripped the platform's own last-admin-protection guard instead of exercising the intended role-revocation path. Fixed by creating a companion admin for the duration of the test.

`f305430` ("Adversarial audit fixes: undocumented CVEs, leaked test credential, stale doc counts") is explicitly described in its own commit message as a second, skeptical re-audit that assumed nothing from the prior pass was correct. It found that the `cryptography` version just bumped to 49.0.0 had its own new high-severity advisory (CVE-2026-69247, a PKCS7 Bleichenbacher oracle), requiring a further bump to 50.0.0, and that `pyasn1` — pulled in transitively via `python-jose` and previously untracked — carried four high-severity DoS advisories on the resolved 0.4.8. It scrubbed a real plaintext test admin/analyst password out of `BUG_TRIAGE.md` , `ENTERPRISE_QA_PHASE1_FINDINGS.md` , and `test-provider-pipeline.sh` (the script now reads `TEST_ADMIN_EMAIL` / `PASSWORD` from the environment instead of hardcoding a credential), and found that `.env.example` 's `JWT_SECRET_KEY` placeholder didn't match `config.py` 's own `Settings` default — a fresh install copying `.env.example` verbatim would boot with a well-known, guessable JWT signing key. It also corrected a stale claim that the scoring engine's test suite had 37 tests; direct count showed 22.

`99db66e` is the direct consequence of `f305430` 's own pyasn1 pin: CI caught, on push, that `python-jose==3.4.0` declares `pyasn1<0.5.0` , directly conflicting with the `pyasn1==0.6.4` pin just added — a real `pip install -r requirements.txt` hard-fails with `ResolutionImpossible` . The commit message is explicit that the prior local verification only ran an incremental `pip install --upgrade` on an already-resolved virtualenv, which doesn't re-validate the full dependency graph the way a fresh install does — "exactly the gap a real CI run exists to catch." Fixed by bumping `python-jose` to 3.5.0, which itself relaxed its own constraint to `pyasn1>=0.5.0` .

`0ba0b1c` root-caused an intermittent "works after reinstall" test flakiness to two divergent, undocumented local virtualenvs ( `backend/.venv` , `backend/.venv_test` ) with no record of which was canonical — `.venv` was missing `cryptography` entirely and had drifted to unpinned latest-on-PyPI versions of `fastapi` / `pytest` / `starlette` . A second, independent root cause was identified: the pinned `asyncpg==0.29.0` has no prebuilt wheel for Python 3.13 on Windows, which had forced whoever set up each venv to work around it differently. The fix bumped `asyncpg` to 0.30.0, deleted the broken `.venv` , and declared `.venv_test` the one canonical dev environment going forward, documented in a new `BACKEND_TEST_ROOT_CAUSE_REPORT.md` . That same report discloses, rather than hides, one pre-existing and still-unfixed issue: cross-file event-loop teardown flakiness in the integration test suite, which does not affect the unit suite CI actually gates on.

## AI Provider Expansion — v0.2.0

`e6539ad` ("feat: expand AI provider ecosystem 6->11 backends, v0.2.0"), recorded 2026-08-19 00:40, is the commit that formally bumped the version from 0.1.0 to 0.2.0 — across `windows/installer.iss` , `linux/debian/control` , `backend/app/main.py` , and `frontend/package.json` . Its actual code change, confirmed by both the commit message and `backend/app/core/runtime_config.py` line 29 ( `AI_BACKENDS = ["ollama", "anthropic", "bedrock", "gemini", "groq", "openai", "kimi", "deepseek", "xai", "mistral", "openrouter"]` ), was adding five new AI backends — Kimi/Moonshot, DeepSeek, xAI/Grok, Mistral AI, and OpenRouter — to the six that already existed (Ollama, Anthropic, Bedrock, Gemini, Groq, OpenAI), following the existing forced-tool-calling client pattern established by `groq_client.py` / `openai_client.py` . Each backend's real base URL and auth format were checked live against the provider's own current documentation rather than assumed: DeepSeek's base URL has no `/v1` segment, unlike every other backend; xAI's error body is a flat `{"code","error"}` shape rather than the nested shape most backends use; Kimi's default model was deliberately set to `kimi-k2.5` rather than one of Moonshot's "thinking" models, which reject a forced tool call with an HTTP 400. The commit deliberately did not add Fireworks AI (its model discovery needs a non-key account ID, a real pattern deviation from every other backend) or Cerebras (a 2-model catalog with no documented error-body schema) — a curated exclusion, not an oversight. The same commit fixed a real, previously-flagged installer packaging bug: `installer.iss` had never excluded `backend/.venv` / `.venv_test` from the bundled files, so every prior Windows installer silently shipped the local dev virtualenv, dropping the installer from roughly 69 MB to roughly 8 MB with no functional change. `637b2ef` followed six minutes later, adding `HORIZON_GRID_AI_PROVIDER_EXPANSION_QA_REPORT.md` to document the new backends' live connection tests, model-discovery fallback lists, and a full regression run reported as 313 tests passed (up from 282).

It's worth being precise about scope here: `CHANGELOG.md` 's own `[0.2.0]` entry additionally lists the deterministic scoring engine, the executive dashboard, two new IOC providers (urlscan.io, Google Safe Browsing), and the visible rebrand to "HORIZON GRID" as part of this release. Checking those claims against git history directly shows the scoring engine and dashboard modules were already present at the Initial commit ( `git log --follow` traces both files back to `9bee96c` , not to `e6539ad` ), and `CHANGELOG.md` itself states elsewhere that "pre-0.2.0 work (initial platform build, GitHub scaffolding, early bug fixes) is captured in the git history itself... rather than a version-numbered changelog entry, since HORIZON GRID's first numbered release is 0.2.0." In other words: v0.2.0 is the point where formal version-numbered release documentation began, and its changelog entry retroactively describes the full state of a build whose scoring engine, dashboard, and README-level branding already existed beforehand; the AI backend expansion (6→11) and the installer virtualenv fix are the specific changes this pair of commits actually shipped.

## Port-Scanner Cancellation Hardening — v0.2.1 and v0.2.2

Six commits across 2026-08-19 evening and 2026-08-20 early morning cover the Security Assessment Toolkit's port scanner.

`da0d692` ("fix: add real cancellation to the Security Assessment port scanner") describes a forensic audit of the full pipeline — frontend, API, validation, the `nmap` subprocess, XML parsing, UI — that found every stage already working correctly against real local targets, with one real gap: once started, a scan could never be stopped short of restarting the backend. It added `POST /api/v1/security-assessment/runs/{run_id}/cancel` and a "Cancel Scan" button, backed by a run-id-keyed dict of live `asyncio.Task` objects so a cancel request can reach an in-flight task and actually kill the OS-level `nmap` process, not just flip a database flag. Two edge cases are called out explicitly in the commit message: `asyncio.CancelledError` does not inherit from `Exception` in Python 3.8+, so it needed its own dedicated `except` branch in both the run orchestrator and the nmap tool's subprocess handling; and a run orphaned by a backend restart (no live task in the current process) still resolves to `CANCELLED` via a direct, rowcount-guarded database write. A migration bug was caught by a genuinely failing test before release — the new `CANCELLED` enum value was first added lowercase, not matching the column's existing uppercase convention — and fixed before the commit. This shipped as v0.2.1.

`5b0edbf` fixed a concurrency race found by adversarial testing: two concurrent cancel requests for the same run could each write their own `security_assessment.run_cancelled` audit row, making one real cancellation look like two in the audit trail. The fix makes the run's own background task the single source of truth for that audit event on the live-task path, while the backend-restart-orphan path still writes its own record, gated on its direct database update having actually changed a row. `1c6ced9` documented this work in `HORIZON_GRID_PORT_SCANNING_QA_REPORT.md` with a verdict of "PORT SCANNING — RELEASE READY."

That verdict was then independently re-checked rather than taken on faith. `9166e75` ("fix: 7 real bugs found by independent re-verification of the cancellation fix") describes nine parallel reviewers re-verifying the v0.2.1 fix from scratch — git archaeology, live browser automation, an adversarial security pass, real concurrency races, and full-application regression — and finding four new defects in the scanner itself plus three unrelated ones surfaced during regression: IPv6 scans never actually probed the target because nmap needs an explicit `-6` flag for an IPv6 literal, and without it silently exits 0 having scanned 0 hosts, reported identically to a real clean scan; an unknown or mistyped scan-profile id was silently accepted and reported as a clean scan with no error anywhere; a malformed CIDR value crashed the run endpoint with a raw HTTP 500; and a run's own successful completion could silently overwrite a status another writer (such as the orphan-recovery sweep) had already finalized. The three unrelated findings were a Spamhaus provider misreading a DNS query-rejection as a positive "malicious" verdict, a confusing generic 401 instead of "Account disabled" on the losing side of a concurrent last-admin-protection race, and an AI-generated assessment that could cite specific findings in prose while leaving its structured supporting-evidence list empty. All seven were fixed, covered by new tests (365 passed, up from 350), and the version was bumped to v0.2.2. `2ef2ad1` is a small CI-only follow-up: the new IPv6 argv tests failed on CI's bare "unit" job, which has no `nmap` binary installed, because the tests' mock didn't also cover `is_available()` — fixed by mocking that too. `2e42a14` documented the nine-reviewer pass as an addendum to the existing QA report, keeping the same "PORT SCANNING — RELEASE READY" verdict but on stronger evidence.

Reading the resulting code directly (`backend/app/core/security_assessment.py`, `backend/app/security_assessment/nmap_tool.py`) confirms these fixes are actually present: `cancel_run()` and a matching `except asyncio.CancelledError` branch in `_execute_run()` exist with the described audit-dedup gating, `nmap_tool.py`'s `run()` conditionally adds `-6` for IPv6 targets, and `start_run()` validates the profile id and catches CIDR-parsing `ValueError`s before any run row is created.

## Mission-Critical Reliability Hardening — v0.2.3

Fourteen commits on 2026-08-20 form the largest single phase in this history: a dedicated review for "unattended remote deployment" — installing the platform once at a remote site with no developer access afterward.

`5491332` is the core hardening commit. It added restart policies, log rotation, and memory limits on all 8 Docker Compose services alongside a new dependency-aware `GET /health/detailed` endpoint (checking Postgres and Redis) next to the existing dependency-free `/health` that CI relies on; boot-time auto-start plus a 5-minute health watchdog on both platforms (Windows Scheduled Tasks, Linux systemd timers); and found, in the process, that Linux's `horizon-grid.service` unit file was correct but had never actually been `systemctl enable`d anywhere in the codebase — a working install would silently not survive a host reboot. It wired the existing SSRF guard (`assert_safe_outbound_url`) into the real Ollama AI-call path, including the `.env`-only fallback singleton used by any wizard-driven install that never touches the runtime-config web UI, and into the config-save path rather than only the Test-Connection convenience endpoints. It fixed a dead-retry-code bug where `BaseProvider.run()` was silently catching the exact exception types the orchestrator's retry loop was meant to see, meaning `provider_max_retries` had no real effect for providers that didn't handle their own network errors. It added fixed-window login brute-force rate limiting keyed by attempted email, a concurrency cap on security-assessment scan execution, and CIDR-size-proportional nmap timeout scaling. Separately, it found that `check-prerequisites.sh`'s `add_check()` function interpolated bash's lowercase `true`/`false` into generated Python source as bare identifiers — Python doesn't recognize those as booleans, so every call silently failed and fell back to a previous value, leaving the script's JSON `checks` array permanently empty (console output and exit code were computed separately and were unaffected, which is why this had gone unnoticed).

`057404e` added real database restore scripts on both platforms — previously only backup scripts existed, and the one documented manual restore procedure was self-contradictory (it told an operator to stop the whole platform, then restore into "the running Postgres container," but stopping the platform stops Postgres too). The new scripts stop only the services that write to the database, leave Postgres running, and require a typed confirmation unless `--force`/`-Force` is passed; this was live-verified end to end (backed up, inserted a marker row, restored, confirmed the marker was gone and the platform came back up correctly). The same commit gave Redis a persistent named volume — confirmed live before the fix that a value set, then a container restart, silently lost it — and fixed `Test-BackendHealth`/`hg_test_backend_health` (and its frontend equivalent) hardcoding port 8000/3000 regardless of a customized `HOST_PORT_BACKEND`/`HOST_PORT_FRONTEND`, meaning every unattended caller (the watchdog, service status, restart, open-platform) had been checking the wrong port forever on any non-default install.

`20fc3cc` fixed a frontend gap where `FinalAssessmentPanel` rendered the identical badge for `ai_outcome="skipped_no_evidence"` (a correct decision) and `ai_outcome="failed"` (a genuine AI generation failure) — the backend already tracked and sent `ai_outcome` in every response, but the frontend's `FinalAssessment` type simply never included the field, so nothing read it. The same commit added a global 10 MB request-body-size limit (checked against `Content-Length` before the body is read) and `max_length` constraints on several previously-unbounded free-text fields (`LookupCreateRequest.value`, case description/note-body fields), none of which had any limit before. `5052813` closed an exposure where `/docs`, `/redoc`, and the raw OpenAPI schema were reachable unconditionally with no environment gate, exposing a full map of every route and request/response shape to anyone who could reach the API — gated behind `settings.environment != "production"` and wired into `docker-compose.prod.yml`.

`6aaffb8` added a new Mission-Critical Operations Manual and corrected two stale claims found in the existing Operations Guide: its container table had claimed only `celery_worker`/`celery_beat` auto-restart (true before this session, wrong afterward — all 8 services do), and its health-check section described an unspecified "backend health endpoint" without noting it now specifically hits `/health/detailed`. `997f6b8` bumped the version to 0.2.3 and rebuilt both installers, installing the Linux `.deb` as a real upgrade over an existing v0.2.1 instance inside a WSL test harness — the reconfigure wizard ran end to end (prerequisite gate, pre-upgrade backup, port auto-remap around still-running old containers, new `.env`, containers healthy, all three systemd units enabled) — while disclosing that a live elevated Windows install was not performed in this run, because it requires an interactive UAC prompt the autonomous session had no way to satisfy; the Windows installer's contents were verified directly from the Inno Setup build log instead. `2702ca0` added the certification report itself, with a final verdict of "MISSION-CRITICAL READY WITH DOCUMENTED LIMITATIONS," covering 17 real gaps found and fixed (2 of them self-discovered during the review rather than flagged by the initial assessment).

Two smaller fixes followed the same day. `d7f13ae` fixed a real crash: the relationship graph's underlying `react-force-graph-2d`/d3-force simulation mutates its input data in place once rendering starts, rewriting `x`/`y`/`vx`/`vy`/`index` onto node objects and replacing each link's string `source`/`target` with direct node-object references — since `RelationshipGraph` fed the same memoized `graphData` object to both the force graph and the list view, switching to "View as list" after the graph had rendered once crashed the whole page with React error #31 ("Objects are not valid as a React child"). The fix gives the force graph its own shallow-cloned copy of the data. `f2b61f9` corrected the certification report's soak-test section, which had been mistakenly reported as "still running" — the real 3-hour, 171-cycle soak test had actually completed, with memory flat across the run and zero spontaneous container restarts.

The final four commits of this phase are documentation reconciliation: `20e5e22` found and corrected real drift — two chapters still described the wizard polling plain `/health` instead of `/health/detailed`, six files claimed no rate limiting existed on `/auth/login` (a real limiter had just been added), and provider/AI-backend counts across a dozen files were stuck at pre-v0.2.0 numbers (16 providers, 5 AI backends) despite urlscan.io, Google Safe Browsing, and six more AI backends never having been folded into the technical reference chapters. `be41309` rebuilt `CHANGELOG.md` from scratch (not appended) covering v0.2.0 through v0.2.3, along with a version audit, release notes, and test-evidence documents. `e27bde5` regenerated every PDF/DOCX and recaptured 19 screenshots (18 refreshed, 1 new) against the live v0.2.3 build, catching a real bug in the process: the documentation build script's cover-page template had "0.1.0" hardcoded as the version stamp on every previously published PDF, regardless of the actual release — now driven from a single version constant. Two final follow-ups closed out the phase after a real CI failure surfaced post-push: `40ee62b` fixed two Ollama SSRF regression tests that depended on resolving `host.docker.internal` via genuine DNS, which only resolves inside a Docker Desktop network and broke GitHub Actions' bare Ubuntu runner — fixed by mocking the DNS-resolution step to a fixed RFC 5737 test address rather than skipping the check — and `f6be98d` recorded that failure and its fix in the test evidence and changelog, consistent with the release's own stated rule against freezing documentation prematurely.

## The threat_assessment Validator Fix

`beb9243` ("fix: reject a bare verdict word in threat_assessment that contradicts final_verdict"), recorded 2026-08-21 04:03, was found live during a full functional-testing pass against a freshly installed instance: investigating `8.8.8.8` with Ollama running `llama3.2:3b` returned `threat_assessment="malicious"` — a bare one-word label, not a sentence — in the same response as a correctly computed `final_verdict="benign"` and `malicious_probability=0.0`. The platform's existing `_verdict_must_agree_with_risk` validator (`backend/app/ai/schemas.py`) only checked `final_verdict` against the numeric risk fields; nothing checked `threat_assessment`'s own text content, and `FinalAssessmentPanel.tsx` renders the two as separately labeled tabs ("Threat Assessment" / "Risk & Verdict"), so an analyst reading either tab alone would see the platform's opposite conclusion.

The fix, read directly from `backend/app/ai/schemas.py` lines 283–309, is a third `model_validator(mode="after")`, `_threat_assessment_bare_verdict_must_agree_with_final_verdict`, deliberately narrow: it only fires when `threat_assessment`, stripped and lowercased, is an exact match for one of `Verdict`'s own string values, and that value's malicious/benign category contradicts `final_verdict`'s. An ordinary narrative sentence that merely contains the word "malicious" mid-sentence never matches this and is unaffected — confirmed by a dedicated regression test asserting exactly that. The validator reuses the same retry-on-`ValidationError` mechanism already in `generate_final_assessment()` rather than introducing new failure-handling logic.

`399c27f` documented this alongside two unrelated P0 installer bugs found during the same interim testing pass — a stale shipped Windows `.exe` missing `docker-compose.yml`, and a leftover verification-harness registry entry silently redirecting installs to a fake path — in `FULL_FUNCTIONAL_TEST_REPORT.md`, explicitly marked "interim, not final," with roughly a third of a 33-phase testing mission covered and the remainder explicitly listed as not yet done. `108d6fa` recorded all three bugs in `CHANGELOG.md`'s v0.2.3 entry along with two new regression tests for the `threat_assessment`/`final_verdict` contradiction — one asserting the exact reproduced payload is rejected, one asserting an ordinary sentence containing the word "malicious" is not.

## Final Visual-Identity and Branding Pass — v0.2.4

The last five commits, on 2026-08-21, form a deliberately scoped, presentation-only pass — no backend logic, database schema, authentication, IOC processing, AI provider logic, provider API logic, port-scanner logic, or admin permission was touched, verified by a full backend regression run and a clean frontend `tsc --noEmit` typecheck both before and after.

`c927bf3` ("feat: original visual identity spine (Datum Signal) for the branding pass") is the foundation commit, made because the product name and tagline were, in the commit's own words, "barely visible anywhere in the running application." It introduced a new HSL color palette (a CRT-phosphor teal-cyan primary deliberately pulled off the generic-SaaS-blue hue most dark-mode dashboards default to, against a near-black shell with a warm off-white foreground); an original, wholly geometric logo mark (`Logo.tsx` — a horizon line with two open "signal" nodes converging on one filled "detection" node, chosen specifically because it depicts the tagline rather than merely looking tactical, with no insignia, seal, shield, or existing company/military symbol); a six-state operational status language (`StatusBadge.tsx`: OPERATIONAL/DEGRADED/WARNING/CRITICAL/OFFLINE/UNKNOWN, each state pairing a distinct hue, fill density, border style, and icon — never color alone); a `BrandHeader.tsx` combining the mark with a live SYSTEM STATUS badge polled from the real `GET /health/detailed`; a three-typeface system (IBM Plex Sans Condensed for headings/wordmark/labels, IBM Plex Mono with tabular numerals for raw technical values, body text left unchanged); a radius-contrast rule (soft outer panels, sharp interactive controls) and flat hairline-bordered cards replacing drop-shadows, both applied once at the shared component level; and a new, additive `/about` page.

`a098765` applied that shared header/nav to the ten pages that compose it individually, swapping each page's own `<TopSearchBar /><WorkspaceNav />` pair for `<BrandHeader />` and restyling section headings — explicitly presentation-only, with no API call, state variable, prop signature, or business logic touched on any of the ten files, verified with a clean typecheck after every edit. `f5fac81` extended the branding to every exported report format: PDF exports gained a real header/footer with page numbering via reportlab's canvas callbacks, CSV exports gained a platform/investigation-ID field pair, and the client-built Markdown export gained a matching header — verified live by running a real investigation, exporting the PDF, and visually confirming the header/footer rendered correctly, which caught and fixed a text-overlap spacing bug the unit tests alone hadn't caught. `ce55ec8` bumped the version to 0.2.4.

`1dba60d` closed out the release by syncing `CHANGELOG.md`, the release notes, and the version audit, and — consistent with the documentation-reconciliation pattern seen throughout this history — found six further stale claims while cross-referencing the doc set: three files still said "five supported AI backends" (should be eleven), one said the Linux wizard only offers five AI backends when its own source lists all eleven, and one still said "16 registry providers" in a figure caption whose surrounding prose had already been corrected to 18. Ten canonical screenshot filenames were overwritten with fresh captures against the live v0.2.4 build, and all 25 PDFs/DOCX were rebuilt with the corrected version stamp.

## Where This Leaves the Project

Across these 36 commits — from the Initial commit on 2026-08-18 through the branding sync on 2026-08-21 — the recurring pattern is not a single clean build but a series of adversarial re-checks, each one assuming the previous pass might be wrong and finding something real when it looked again: a second audit catching a new CVE the first audit's own fix introduced, nine independent reviewers re-verifying a "release ready" scanner fix and finding seven more bugs, a live investigation of `8.8.8.8` catching an AI self-contradiction no existing validator checked for. The most recent development-cycle document in this repository, `FULL_FUNCTIONAL_TEST_REPORT.md` (2026-08-21, cited above), states its own verdict as interim rather than final, with roughly two-thirds of its planned 33-phase testing mission explicitly marked as not yet run — a disclosed gap, not a claimed one, consistent with how every phase in this history has been documented.

## Bug History: What Actually Broke, and What Got Fixed

This project's own QA and hardening reports are written as adversarial, live-system audits rather than marketing copy, and several of them explicitly disqualify the product from a "release ready" verdict at the time they were written. That candor is treated here as material worth keeping, not smoothing over. This section walks through five phases of the project's development — early hardening, the AI-backend expansion, the Security Assessment Toolkit / port scanner, the mission-critical reliability pass, and the final release cycle — and lists the real bugs each phase's own reports document, including several that were found and disclosed but never fixed. Every row below is sourced from a specific QA/report document or a specific source file read directly; where a claim could not be independently re-verified against code, that is stated rather than implied.

### Early Hardening

The earliest hardening work covered in the project's reports is not glamorous: two divergent, undocumented backend virtualenvs, a broken credential-test flow in the Windows Setup Wizard, a `.env` writer with no escaping, and a first pass at multi-admin RBAC and database-backed runtime provider configuration. Threaded through this phase is `ENTERPRISE_QA_PHASE1_FINDINGS.md`, a large adversarial red-team pass run directly against the live, running stack — it is explicitly an audit report, not a fix report, and states plainly that it found problems without fixing them. Only a representative subset of its 74 findings is reproduced below; the rest remain in that document, disclosed and open at the time it was written.

| Bug | Impact | Root Cause | Fix | Status |
|---|---|---|---|---|
| Two divergent, drifted backend virtualenvs (`backend/.venv`, `backend/.venv_test`) | `pytest app/tests/unit` gave 4 collection errors, 0 tests run (`ModuleNotFoundError: No module named 'cryptography'`) | No documented canonical venv; `asyncpg==0.29.0` had no Windows/Python-3.13 wheel, forcing ad hoc workarounds per venv | Deleted `backend/.venv`; bumped `asyncpg` to `0.30.0`; reinstalled `.venv_test` from `requirements.txt`; declared it canonical in `docs/TESTING.md` | Fixed and verified — 282/282 unit tests passing, 5 consecutive clean runs |
| Fresh `docker compose up -d postgres redis` leaves an empty DB schema | Running the suite without a manual `alembic upgrade head` produced 36 failed + 27 errors, all "relation does not exist" — looked like an app bug but was a missing setup step | Migrations only auto-run when the `backend` service itself starts | Documented the exact `alembic upgrade head` command in `docs/TESTING.md` | Fixed (documentation only) |
| Setup Wizard "Test Connection" required a session token only ever set inside the "Start Installation" handler | On a reconfigure run, no login ever happened, so every Test Connection click failed with a misleading "create the administrator account first" error even though the account already existed | Token-acquisition logic lived only in the install-click handler, not as an independent path | New `Get-OrCreateWizardSession` (login-only, never registers an account), called directly from each Test Connection click | Fixed |
| `Write-EnvFile.ps1` wrote `.env` values with no escaping | A `#` in a password/API key silently truncated the rest of the line; embedded newlines could corrupt the file | No escaping/validation before writing values to `.env` | New `ConvertTo-SafeEnvValue` strips CR/LF and rejects embedded `#` | Fixed |
| Hybrid Analysis provider returned HTTP 301 instead of a valid result | Real Hybrid Analysis lookups were silently broken, not just the test button | `httpx` doesn't follow redirects by default; `hybrid-analysis.com` now permanently redirects to the bare domain | Fixed in `app/providers/connection_test.py` and `app/providers/stubs/hybrid_analysis.py`, plus the unit-test mock URL | Fixed and confirmed live (243ms, correct `no_data` for a test hash) |
| Admin-initiated password reset left every already-issued access/refresh token valid | The opposite of the intended security effect — a compromised session stayed alive after "resetting" the password, for up to 7 days | JWTs are stateless (signature + expiry only); nothing tied token validity to password state | Added `token_version` to `User`; `get_current_user()` and `POST /auth/refresh` reject any token whose embedded version doesn't match the DB value; `reset_password()` increments it | Fixed, verified live and by automated test |
| `/auth/me` always returned `full_name: ""` | Display bug regardless of stored value | Found and fixed as part of the RBAC work | Corrected to return the stored value | Fixed |
| 2 pre-existing tests broke against the new async, tuple-returning `_get_ai_client()` signature | Regression in the runtime-provider work | Signature change not reflected in test doubles | Caught and fixed in the same session's regression pass | Fixed — 153/153 passing afterward |
| Oversized IOC value (e.g. a long URL) crashes the `/lookup/stream` SSE generator, leaving the investigation stuck at `status='running'` forever | Any user with ordinary `lookup:create` permission can corrupt a lookup into an unrecoverable state by pasting a long value | OTX's `source_url` has no length cap and can exceed the `provider_results.source_url VARCHAR(2048)` column; the generator's exception handler then does a second `commit()` on an already-rolled-back session, raising an uncaught `PendingRollbackError` that escapes the "mark FAILED" safety net entirely | None found in this report | Open (disclosed, not fixed) |
| No request-correlation ID anywhere in the backend | Concurrent operations are unattributable from `docker logs` alone | `app/main.py` registers only `CORSMiddleware` and the Prometheus instrumentator; no request-ID middleware anywhere | None found in this report | Open (disclosed, not fixed) |
| Failed logins are invisible in stdout logs beyond a bare `401` | Security-relevant events not surfaced in ordinary log inspection | `login()` in `app/api/routes/auth.py` has zero `logger.*` calls | None found in this report | Open (disclosed, not fixed) |
| Login response time leaks account existence via a ~26x timing side-channel | Existing-email+wrong-password median 231.4ms vs. nonexistent-email median 8.8ms (N=40/arm) | `if not user or not verify_password(...)` short-circuits — bcrypt only runs for real emails | None found in this report | Open (disclosed, not fixed) |
| No rate limiting/backoff/lockout on `/auth/login` | 15 rapid failed logins all returned 401 with no throttling signal | `lookup.py` has an explicit `RateLimiter`; `/auth/login` does not | None found in this report | Open (disclosed, not fixed) |
| Weak, hardcoded default DB credentials baked into `backend/app/core/config.py` / `docker-compose.yml` | Any deployment that skips the wizard's random-secret generation and runs bare `docker compose up` is exposed | Default `postgresql+asyncpg://ioc:ioc@postgres:5432/ioc_intel`; live-confirmed the running Postgres instance actually used this default | None found in this report; mitigated only by the wizard, not the app itself | Open (disclosed, not fixed) |
| `jwt_secret_key` has a hardcoded fallback (`"change-me-in-production"`) with no startup guard | Same string signs every JWT and, via HKDF, derives the at-rest credential-encryption key | No code-level safety net for a deployment path that skips the wizard | None found in this report; confirmed not the live value in practice because the wizard generates a real secret | Open (disclosed, not fixed) |

The QA phase-1 report also discloses a large number of P2/P3/P4 findings not reproduced in the table above — among them a stale AI-provider "last test: failed" badge that can persist on a healthy backend, `mitre_mappings` coming back empty for a textbook Log4Shell case, an unhandled `IntegrityError` on concurrent duplicate "Add to Basket" requests, CSV/PDF export-injection issues (carried forward and addressed later — see the Final Release phase below), and a `pip-audit` false negative against packages independently confirmed vulnerable via direct OSV.dev queries. All of these are recorded in `ENTERPRISE_QA_PHASE1_FINDINGS.md` as open at the time of writing; this document does not claim any of them were fixed within that report.

## AI Backend Expansion

The provider registry for AI backends ( `backend/app/core/runtime_config.py` line 29) lists 11 entries: `ollama` , `anthropic` , `bedrock` , `gemini` , `groq` , `openai` , `kimi` , `deepseek` , `xai` , `mistral` , `openrouter` . Per `HORIZON_GRID_AI_PROVIDER_EXPANSION_QA_REPORT.md` , six of those (Ollama, Anthropic, Bedrock, Gemini, Groq, OpenAI) predate this expansion; five (Kimi, DeepSeek, xAI/Grok, Mistral, OpenRouter) were added in this v0.2.0 pass, each reportedly given a live base-URL check, forced `tool_choice` verification, live model discovery, and a dedicated batch of new unit tests. The regression suite is reported at 313 passed (282 pre-existing + 31 new). None of the live HTTP timing figures, the 313-test total, or the installer hash claims in that report were independently reproduced this turn — they are relayed as the report's own claims, not independently confirmed.

The report's own "fixed defects" section lists exactly three bugs found during this pass, plus one carried-forward item explicitly flagged as not new to this cycle:

| Bug | Impact | Root Cause | Fix | Status |
|---|---|---|---|---|
| `python-jose 3.4.0` + `pyasn1 0.6.4` pip `ResolutionImpossible` | Dependency install failure | Version conflict between the two pinned packages | Resolved in the immediately preceding backend-test-stability cycle; re-verified still holding in this release | Fixed (carried forward, not new to this pass) |
| Windows installer bundled the local dev virtualenv | The installer bloated from ~69 MB to bundling ~190 MB of unused dev-venv content | `installer.iss` never excluded `backend/.venv` / `.venv_test` | Excluded the dev venv directories from the installer build | Fixed |
| `standalone-ai-guide.md` never documented the OpenAI backend | OpenAI, added as the 6th backend in an earlier cycle, had no user-facing documentation until this pass | Documentation not updated when OpenAI was originally added | Documentation updated during this pass | Fixed |

Two further items are recorded as corrections rather than bugs: an earlier, narrower fetch of a Mistral guide page had incorrectly suggested named-tool forcing wasn't supported, corrected against the authoritative API reference; and a documented (not fixed, because it is not a defect) Moonshot/Kimi API behavior — forcing `tool_choice` conflicts with Kimi's "thinking" models ( `kimi-k3` , `kimi-k2.7-code` ), returning HTTP 400 — was worked around by choosing `kimi-k2.5` as the default model, with a dedicated test ( `test_kimi_400_thinking_conflict_is_reported_clearly` ) asserting the error surfaces clearly rather than silently. `backend/app/ai/service.py` contains inline comments describing several other real, previously-fixed bugs (a Spamhaus `"listed": false` misread, an NVD-field-length prompt-pollution issue, a self-contradictory verdict/probability bug, an EICAR-hash fabrication bug), but nothing in the expansion QA report ties those specifically to the 6→11 backend expansion, so they are not attributed to this phase here.

## Security Assessment Toolkit (Port Scanner)

The Security Assessment Toolkit wraps `nmap` behind three fixed scan profiles ( `quick` , `standard` , `web` ) with no caller-constructed argv strings — targets are passed as a single, always-terminal `asyncio.create_subprocess_exec` argument, never through a shell ( `backend/app/security_assessment/nmap_tool.py` ). Two dedicated QA reports, `HORIZON_GRID_PORT_SCANNING_QA_REPORT.md` and `SECURITY_ASSESSMENT_QA_REPORT.md` , document a cluster of real bugs found while building cancellation support and hardening this feature across versions v0.2.1 and v0.2.2. A subset of these — the cancellation flow itself, the IPv6 `-6` flag fix, the profile-id validation, the malformed-CIDR handling, and the completion-status race guard — were independently confirmed this turn by reading `backend/app/security_assessment/nmap_tool.py` and `backend/app/core/security_assessment.py` directly; the remainder are relayed as the QA reports' own claims, marked accordingly.

| Bug | Impact | Root Cause | Fix | Status |
|---|---|---|---|---|
| No way to cancel a running scan | A slow or regretted scan could only be waited out; restarting the backend to escape it would orphan the running `nmap` process | No cancel endpoint, button, or subprocess-kill path existed at all | Added `CANCELLED` status, a cancel endpoint, a task-tracking dict, dedicated `CancelledError` handling in both the orchestrator and the nmap tool, and restart-resilience cleanup | Fixed (v0.2.1) — confirmed present in `security_assessment.py` ( `cancel_run()` , `_execute_run()` ) and `nmap_tool.py` 's `except asyncio.CancelledError` branch |
| IPv6 scans never actually probed the target | High impact — IPv6 scanning was silently non-functional; nmap exited 0 having scanned 0 hosts, indistinguishable from a real clean scan | `nmap` needs an explicit `-6` flag for an IPv6 literal; it was missing | Added `-6` conditionally for IPv6 targets | Fixed (v0.2.2, found by an independent re-verification pass) — confirmed present at `nmap_tool.py` lines 124–131 |
| Migration added the enum value `'cancelled'` lowercase against Postgres's existing uppercase enum labels | Two tests failed with `invalid input value for enum securityassessmentrunstatus: "CANCELLED"` | Case mismatch between the new migration and existing enum labels | Changed to `ADD VALUE IF NOT EXISTS 'CANCELLED'` | Fixed |
| Two concurrent cancel requests for the same run could each write their own audit row | Audit trail showed a single cancellation as two events (audit-accuracy issue, not a functional one) | No single designated writer for the audit event on the live-task cancellation path | Made the run's own background task the sole writer for the live-task path; the restart-orphan path gates its write on rowcount | Fixed — confirmed present at `security_assessment.py` 's `cancel_run()` and `_execute_run()` |
| An unknown or mistyped scan profile id was silently accepted and reported as a successful, empty scan | High impact — indistinguishable from a real clean scan, with `error_message: null` | Profile id not validated before creating the run row | Validate the profile id against the tool's known profiles up front, returning 400 | Fixed — confirmed present at `security_assessment.py` lines 298–313 |
| A malformed CIDR lookup value crashed the run endpoint with a raw 500 | Medium impact — unhandled server error on bad input | `ipaddress` parsing raised an uncaught `ValueError` | Catch the `ValueError` , return 400 | Fixed — confirmed present at `security_assessment.py` lines 260–266 |
| A run's own successful completion could clobber a status another writer had already finalized | Medium impact — a run could end up `COMPLETED` while still carrying a stale `FAILED` error message | No guard against a race between the run's own completion write and a concurrent finalizer (e.g. startup orphan-recovery) | Guard the completion status update with `WHERE status IN (PENDING, RUNNING)` | Fixed — confirmed present at `security_assessment.py` lines 462–474 |
| Empty `tool_ids` was accepted, producing a no-op run | Low impact | No validation on an empty tool selection | Reject at the schema level (422) | Fixed per the report; not independently verified in the schema file this turn |
| Spamhaus provider misreported a DNS query-rejection as a positive "malicious" verdict | A benign domain (e.g. `example.org` ) could be flagged malicious for no real reason | Query-rejection response mishandled as a positive listing | Correctly report `unknown` on rejection | Fixed per the report (found incidentally during regression, unrelated to scanning); not independently verified this turn |
| Confusing generic 401 instead of 403 "Account disabled" on the losing side of a concurrent last-admin-protection race | Misleading error on an edge-case concurrency path | Race condition in last-admin-protection logic | Corrected error response | Fixed per the report; not independently verified this turn |
| An AI-generated assessment could cite findings in prose while leaving the structured evidence list empty | UI clickable-citation feature broken for that case | No schema validator enforcing citation/evidence-list agreement | New schema validator using the existing retry-on-validation-failure mechanism | Fixed per the report; not independently verified this turn |
| Two new regression tests failed in the CI "unit" job (no `nmap` binary there) | CI-only test failure | Mock didn't cover `is_available()` | Mock `is_available()` too | Fixed per the report |
| Test-isolation race: `SecurityAssessmentRun.status` flips to `COMPLETED` before the post-completion AI-refresh step finishes | Flaky/corrupting test suite — a test polling only the status field could tear down its shared DB engine mid-refresh | No real synchronization primitive between test assertions and the in-flight background task | Added `wait_for_background_runs()` , awaiting the actual in-flight tasks | Fixed — confirmed present at `security_assessment.py` lines 168–177 |

Two disclosed, non-bug design gaps from this phase are worth carrying forward for completeness: `POST /run` accepts one shared `profile` string for every selected tool, but only `nmap` defines more than one profile, so selecting multiple tools with an nmap-specific profile like `"quick"` would make every non-nmap tool reject it — handled in the frontend by only offering profile ids valid across the intersection of every selected tool's profile set, per the report (not independently verified in the frontend code this turn). Separately, `SECURITY_ASSESSMENT_QA_REPORT.md` and `HORIZON_GRID_PORT_SCANNING_QA_REPORT.md` both disclose that no live elevated Windows installer run or live Linux install/systemd run was performed for either version tested, and that TLS-downgrade testing and DNS subdomain enumeration remain out of scope by design.

## Mission-Critical Hardening

`MISSION_CRITICAL_CERTIFICATION_REPORT.md` certifies v0.2.3 specifically, and states the version number explicitly at line 4. Its stated purpose was unattended-remote-deployment reliability: installer gating, restart/reboot recovery, automatic service recovery, health checks, backup/restore, and a 3-hour, 171-cycle soak test against the live stack (memory reported flat throughout, zero spontaneous container restarts). Two other files in this same research bundle, `LAN_DEPLOYMENT_QA_REPORT.md` and `LAN_REGRESSION_TEST_REPORT.md`, are dated earlier (2026-08-12/14) and cover a separate, distinct effort — moving the platform from localhost-only to LAN accessibility — and carry no version number of their own; one bug below is sourced from that separate work and is labeled as such rather than folded into v0.2.3.

| Bug | Impact | Root Cause | Fix | Status |
|---|---|---|---|---|
| Linux `horizon-grid.service` systemd unit was defined but never actually `systemctl enable`d anywhere in the codebase | A working install would silently not survive a host reboot | The `[Install]` section declared `WantedBy=multi-user.target`, but only `daemon-reload` ran in `postinst` — `enable` was never called | Added the enable call to the wizard's success path | Fixed and verified live via `systemctl is-enabled` → `enabled` |
| `check-prerequisites.sh`'s JSON output had a permanently empty `checks` array | Structured output from the prerequisite gate was silently useless (console output and exit code were unaffected, computed separately, which is why it went unnoticed) | Bash's lowercase `true` / `false` were interpolated as bare Python identifiers, causing a silent `NameError` on every call, swallowed by an error fallback | Passed fields through the environment instead of interpolating into generated source | Fixed; verified live for both pass and simulated-failure cases |
| `BaseProvider.run()` swallowed exception types needed by the orchestrator's retry loop | `provider_max_retries` had no effect for providers without their own network-error handling | Exception handling too broad/incorrect at the base-provider layer | Corrected exception handling to let retryable errors propagate to the orchestrator's retry loop | Fixed, live-verified with a fake provider showing a `ConnectError` now genuinely retried |
| Final Assessment panel showed an identical badge for a genuine AI failure and a correct no-evidence skip | Users could not distinguish "AI backend is down" from "there was nothing to assess" | Backend already tracked/sent `ai_outcome`; the frontend never read it | Frontend updated to read and render `ai_outcome` | Fixed, confirmed against a real `ai_outcome="failed"` record |
| `docker-compose.yml`'s `PUBLIC_API_URL` fallback used `${PUBLIC_API_URL:-http://localhost:8000}` (colon-hyphen) | Fell back to the hardcoded default even when the variable was explicitly set to empty | Colon-hyphen bash/compose substitution treats an explicitly-empty value the same as unset | Changed to `${PUBLIC_API_URL-}` | Fixed — from the separate LAN-deployment work (`LAN_DEPLOYMENT_QA_REPORT.md`), not part of v0.2.3 itself |
| A full-suite regression run reported 365 passed, matching the previous known-good count rather than reflecting the session's actual changes | Test methodology error — the suite had validated a stale artifact, not the current code | The `hgmc` test container was running a stale Docker image predating the session's backend changes | Rebuilt the image, re-ran the suite for real | Fixed — 380 passed after rebuild |
| Redis did not persist across restarts | Cache/session state lost on every restart | Not detailed beyond the fact itself in the report | Redis now persists across restarts | Fixed, confirmed live |

The certification report is explicit about what it did not fix. Disclosed and left open at the time of writing: no automated host-disk-space alerting; no retention/cleanup job for growing investigation tables; Neo4j and OpenSearch fully provisioned with zero actual read/write traffic (an open resource-cost question, not a bug); a DNS-rebinding TOCTOU gap on the SSRF check (it validates a DNS snapshot, but the real outbound call re-resolves independently); no off-site/off-host backup copy; no global ONLINE/LIMITED/OFFLINE UI indicator; no real simulated power-loss test performed; per-subprocess (nmap) OS-level resource limits remain container-level only; `structlog` is configured but unused anywhere in application code; the full ~40-document existing doc set was not exhaustively regenerated; and a live elevated Windows installer run was not performed in this session — the report gives the specific, disclosed reason that it required an interactive UAC prompt the autonomous session could not satisfy. The report's own stated verdict is "**MISSION-CRITICAL READY WITH DOCUMENTED LIMITATIONS**," explicitly not an unqualified "ready" because of that limitations list, and explicitly not "not ready" because every one of the gaps it says it found in its own review was fixed and verified.

## Final Release

The final-release phase in this bundle spans at least two distinct product states and dates, and the six source files should not be read as one continuous story. `FINAL_QA_REPORT.md` (2026-08-14, IOC Intelligence Platform) is a red-team regression pass that states a top-line verdict of **NOT RELEASE READY**, citing a credential-wipe bug, an apparently non-functional export feature, and plaintext secrets exposure as each independently disqualifying per that mission's own severity rules. `RELEASE_BLOCKER_TRACKER.md` tracks the same bugs through fix–test–break–test-again cycles for the same day. `FINAL_SHIP_AUDIT_REPORT.md`, also 2026-08-14, is a separate release-candidate lock/audit (not a fresh QA pass) that reaches **RELEASE READY WITH KNOWN LIMITATIONS**, with a same-day addendum that closed two of its own disclosed limitations outright. `FINAL_RELEASE_QA_REPORT.md` documents a later, separate effort — the "HORIZON GRID" rebrand — also concluding **RELEASE READY WITH KNOWN LIMITATIONS**. The most recent file by timestamp, `FULL_FUNCTIONAL_TEST_REPORT.md` (2026-08-21, v0.2.3), is explicitly marked "Status: IN PROGRESS" and states its own verdict is interim, not final, covering roughly a third of a planned 33-phase mission. No file in this bundle reconciles these into one single authoritative final verdict for the product as a whole.

| Bug | Impact | Root Cause | Fix | Status |
|---|---|---|---|---|
| Credential-wipe-on-save (BUG-01) | Saving certain configuration silently wiped stored credentials | Not detailed beyond the bug identifier in this bundle's summary of the report | First round of fixes was found by adversarial re-check to have gaps; a second round added row-locking and real DB-backed regression tests | Fixed ( `FINAL_QA_REPORT.md` → `RELEASE_BLOCKER_TRACKER.md` ), required two rounds |
| Silent AI-assessment failure on a self-contradictory verdict (BUG-02) | An AI-generated assessment could fail silently rather than surfacing an error | Not detailed beyond the bug identifier in this bundle's summary of the report | Fixed in the same cycle | Fixed ( `RELEASE_BLOCKER_TRACKER.md` ) |
| Unbounded provider data burying severity signal in the AI prompt (BUG-03) | Severity signal could be lost in an oversized prompt | Not detailed beyond the bug identifier in this bundle's summary of the report | Fix caused two further regressions in its own correction rounds: first broke MITRE ATT&CK's long descriptions, then broke Groq's request-size limit; both were subsequently corrected | Fixed, after two additional correction rounds |
| Export feature reported as apparently completely non-functional (BUG-04) | Would have been release-blocking if true | Original test methodology was wrong — the JSON/Markdown export path is client-side and never hit the backend route being tested | N/A — retracted | Retracted, not a real bug — only PDF/CSV correctly 404'd, which was already-documented, non-regression behavior at that time |
| PDF/CSV export not implemented | Two export formats unavailable at initial ship-audit time | Feature not yet built | Implemented in a same-day addendum to `FINAL_SHIP_AUDIT_REPORT.md` ; new installer build produced (180 tests passing) | Fixed (same-day addendum) |
| A narrow first-time-configure concurrency edge case in the credential row-lock fix | Edge-case race during first-time configuration | Related to the row-locking fix applied for BUG-01 | Closed in the same same-day addendum | Fixed (same-day addendum) |
| Windows installer password-mismatch crash | Real installer failure during the HORIZON GRID rebrand cycle | Not detailed beyond the fact of the fix in this bundle's summary | Fixed as part of `FINAL_RELEASE_QA_REPORT.md` 's work, with a user-confirmed real installer run | Fixed |
| Stale shipped `.exe` missing `docker-compose.yml` (P0) | Installer would fail to bring up the stack correctly | Packaging step did not include the compose file in the shipped installer | Not detailed further in this bundle beyond the fact that it was found and fixed | Fixed ( `FULL_FUNCTIONAL_TEST_REPORT.md` ) |
| Leftover registry `InstallLocation` from an old test harness silently redirecting installs to a fake path (P0) | Installs could be silently redirected to an incorrect location | Stale registry key left behind by a prior test harness | Not detailed further in this bundle beyond the fact that it was found and fixed | Fixed ( `FULL_FUNCTIONAL_TEST_REPORT.md` ) |
| Free-text `threat_assessment` field able to contradict the validated `final_verdict` (P1) | An AI-generated assessment's narrative text could disagree with its own structured verdict | No validator enforcing agreement between the free-text field and the structured verdict at the time this was found | Not detailed further in this bundle beyond the fact that it was found and fixed | Fixed ( `FULL_FUNCTIONAL_TEST_REPORT.md` ) |

Disclosed limitations from this phase, not treated as blockers by the reports that state them: Groq's free/shared-tier rate limit (12,000 tokens/minute) is a real external constraint, handled cleanly rather than retried into a storm; AI-generated verdict quality is model-dependent, most visibly with smaller local models; a live plaintext-secrets exposure (including an AWS key pair and several threat-intel provider keys) was found in a dev-environment `.env` file and flagged for rotation, but was not itself rotated by the QA process; and a disposable QA test account and test case were flagged for removal before handing the environment to a real end user. `FULL_FUNCTIONAL_TEST_REPORT.md` additionally lists nine planned phases — log rotation, performance under load, a broader security/RBAC sweep, a Windows reboot test (deliberately deferred), a Linux install-and-reboot test, uninstall/reinstall, full regression, a soak test, and a final post-fix repeat of the critical path — as explicitly not yet run rather than silently assumed to pass, consistent with its own stated interim, not final, verdict.

# Roadmap and Conclusion

### What's Provisioned but Not Yet Wired Up

Two pieces of infrastructure already run in the Docker Compose stack with real configuration fields but no consuming code yet, confirmed directly against source: **Neo4j** (the `neo4j:5-community` container with the APOC plugin, plus `neo4j_uri` / `neo4j_user` / `neo4j_password` settings — no driver instantiation or Cypher code exists anywhere in the backend today; the correlation graph the product actually shows lives entirely in Postgres) and **OpenSearch** (the `opensearchproject/opensearch:2.17.0` container plus an `opensearch_url` setting — no indexing or query code exists against it today). Completing either is a bounded extension of infrastructure already provisioned, not a new subsystem. The full detail, including a diagram of today's actual wiring versus the provisioned-but-unconsumed pieces, lives in the Future Roadmap chapter of the Technical Documentation.

### What's Genuinely Open, Disclosed by the Project's Own QA Reports

This submission's Bug History and Testing Journey chapters carry forward, rather than hide, what the project's own most recent QA reports state as still open at the time they were written: no automated host-disk-space alerting or investigation-table retention job; a DNS-rebinding TOCTOU gap on the outbound SSRF check; no off-site backup copy; several P2–P4 findings from the adversarial red-team pass not yet independently re-verified a second time; and the most recent functional-test report explicitly marking its own verdict as interim, with roughly two-thirds of a planned 33-phase mission not yet run. None of this is softened in the narrative chapters — the project's engineering discipline throughout has been to disclose a gap the moment it's found, not to wait until it's closed to mention it.

### Where the Architecture Is Already Positioned to Grow

The registry pattern behind both the 18-provider IOC ecosystem and the 11-backend AI layer already normalizes heterogeneous sources behind one shared interface each (`BaseProvider` / `supports(ioc_type)`, and `call_claude_json()` respectively) — onboarding further providers or AI backends is proven, incremental work at this point, not a redesign. The same is true of the Security Assessment Toolkit's tool-adapter pattern and the deterministic scoring engine's versioned weight table, both built to be extended rather than rewritten.

### Conclusion

HORIZON GRID's central bet is that a threat-intelligence tool earns trust by showing its work: a deterministic score computed the same way every time, an AI that narrates a number it cannot override, a Provider Health page that reports Unknown rather than faking Healthy, and an Executive Dashboard where every tile is a real query result. That bet shaped the architecture from the scoring engine outward, and it shaped how this submission itself was written — every bug in the Bug History chapter is real, every open item in the Known Limitations sections is still open, and every screenshot in this package is a live capture of the running product, not a mockup.

The name and the tagline were chosen to describe exactly that outcome, not just to sound serious: **HORIZON GRID — Every Signal. One Operational Picture.**